

CH4 - Clip 5.

Container 이론



01

Container란?

02

Container 주요 구성요소 & 필요성

03

Container 장단점

04

Container 실제 사례

05

MLOps에서의 Container 인프라

06

Docker

1. Container란?

Container란?



Container란?

Container?

- 어떤 것을 실행하기 위해 필요한 모든 요소를 포함하는 소프트웨어 패키지
- 운영체제를 가상화한 형식이기 때문에, 클라우드에서부터 개인 노트북에 이르기까지 어디서나 실행이 가능
- SW 개발팀은 컨테이너화를 통해 더욱 신속하게 작업을 진행하고 효율적으로 소프트웨어 배포할 수 있음 => 전례없는 수준의 확장성을 확보하게 됨

Container란?

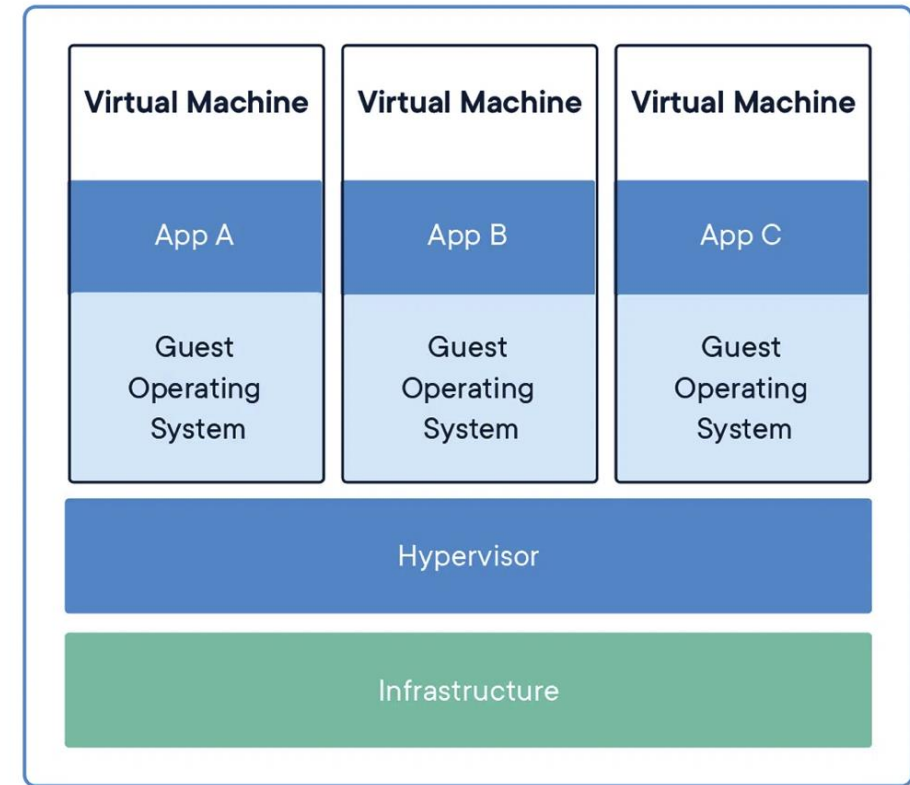
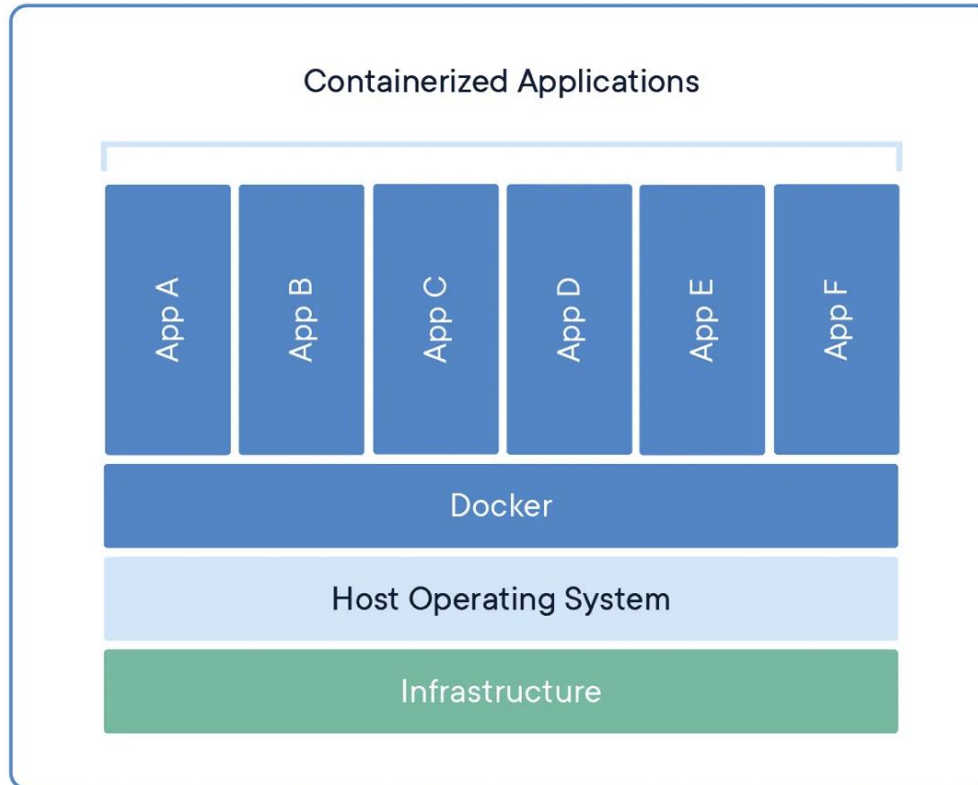
Container 개념

- 기본 정의 : Container 는 애플리케이션과 그 필요한 모든 것(코드, 런타임, 시스템 도구, 시스템 라이브러리 등)을 포함하는 표준화된 단위 .
애플리케이션은 어떤 컴퓨팅 환경에서도 일관되게 실행될 수 있음
- 작동 원리 : Container는 운영 체제의 커널을 공유하며, 각 컨테이너는 서로 격리된 공간에서 실행. 이는 각 Container가 자체 파일 시스템을 가지고 있는 가벼운 '가상 환경'과 유사하다고 보면 된다.

Container와 Virtual Machine 차이

- Container와 Virtual Machine은 리소스 격리와 할당의 이점은 비슷하지만 Container가 하드웨어 대신 운영체제를 가상화하기 때문에 기능이 다릅니다.
Container는 더 휴대가 쉽고 효율적
- Virtual machine : 전체 가상화를 통해 각 VM에는 전체 운영 체제와 물리적 하드웨어 구조도 포함. 이는 추가적인 오버헤드를 발생시키며 리소스 사용이 많음
- Container : 반면에 컨테이너는 운영 체제 수준의 가상화를 사용하여 훨씬 가볍고, 필요한 컴포넌트만 포함 (하드웨어 구조는 가상화하지 않음)

Container란?



Container란?

Container 특징

- 독립된 실행 환경 : 애플리케이션과 그 종속성을 담은 격리된 환경
- 경량화 : 전체 Os를 가상화하지 않고 필요한 컴포넌트만 포함
- 이미지 기반 : 애플리케이션 실행에 필요한 모든 파일을 포함하는 이미지 기반

2. Container 주요 구성 요소

Container 주요 구성 요소

주요 구성 요소

- Image
- Registry
- Container Runtime

Container 주요 구성 요소

Image

- Container Image는 애플리케이션과 그 실행에 필요한 모든 파일을 포함한 불변의 템플릿
- 예시 : Docker 이미지는 Dockerfile로 정의되며, 애플리케이션의 코드/라이브러리/종속성/환경 설정 등을 포함

Registry

- Container Image가 저장되고 공유되는 곳입니다.
- 예시 : Docker Hub, Google Container Registry 등이 있으며, 이곳에서 Image를 다운로드하거나 업로드할 수 있습니다.

Container Runtime

- Container를 실행하기 위한 환경을 제공합니다.
- 예시 : Docker, rkt 등이 있으며, 이를 통해 Container image를 실행하여 Container Instance를 생성

Container 필요성

환경 일관성 제공

- Container는 개발, 테스트, 생산 환경에서 동일한 소프트웨어 환경을 유지할 수 있게 해 줍니다. 이는 애플리케이션의 이식성을 증가시키고 환경에 따른 문제를 줄입니다.

자원 효율성 증가

- Container는 VM(가상 머신)보다 적은 리소스를 사용하여 더 많은 애플리케이션을 호스트할 수 있게 해 줍니다. 이는 하드웨어 비용을 줄이고, 자원 사용률을 최적화합니다.

배포 및 확장 용이성

- Container는 배포와 확장이 용이하며, 빠른 시작과 중단이 가능합니다. 이는 CI/CD 파이프라인과 잘 통합되어 빠른 개발 및 배포를 가능하게 합니다.

3. Container 장단점

Container 장점

경량성 및 빠른 시작

- 운영 체제 전체를 부팅할 필요가 없기 때문에 Container는 빠르게 시작할 수 있음
- Linux, Windows, Mac 등의 운영체제를 가리지 않고, 가상머신/물리적 서버/개발자 컴퓨터 등의 사실상 어느 환경에서나 구동가능

일관된 환경

- 개발부터 운영까지 동일한 환경을 유지하여, '개발에서는 잘 되는데, 운영 환경에서는 안 된다'는 문제를 해결

효율적인 자원 사용

- Container는 운영 체제의 커널을 공유하므로, 가상 머신에 비해 더 적은 양의 메모리와 CPU를 사용

격리와 분리

- 책임을 깔끔하게 분리. 개발자는 애플리케이션의 로직과 종속 항목에 집중하고, IT 운영팀은 배포 및 관리에 집중
- 다른 애플리케이션으로부터 논리적으로 격리된 OS 환경을 제공

Container 단점

보안 취약성

- VM보다 격리 수준이 낮을 수 있으므로 보안 취약성이 있을 수 있음

관리 복잡성

- 다수의 컨테이너를 효과적으로 관리하기 위해서는 복잡성이 증가 => Orchestration 도구 필요

4. Container 활용 사례

멀티 클라우드 및 하이브리드 클라우드 환경에서 활용

사례

- 다양한 클라우드 환경에서 머신러닝 모델을 운영하기 위한 Container 사용

구체적 내용

- **문제 상황:** 다양한 클라우드 제공업체의 서비스를 사용하여 머신러닝 모델을 배포 및 관리하는 복잡성.
- **해결 방법:** Docker와 같은 Container를 사용하여 여러 클라우드 환경에서의 일관된 운영 및 관리.
- **결과:** 클라우드 제공업체에 관계없이 일관된 방식으로 머신러닝 모델을 배포 및 관리, 운영 비용 절감 및 확장성 향상.

대규모 머신러닝 파이프라인 최적화

사례

- 대규모 데이터 처리와 머신러닝 모델 학습을 위한 파이프라인에서 Container 사용

구체적 내용

- **문제 상황:** 복잡한 데이터 파이프라인과 다양한 머신러닝 모델이 다수의 서버와 환경에서 관리되어야 하는 상황.
- **해결 방법:** Docker 컨테이너를 사용하여 모든 데이터 파이프라인 구성 요소와 머신러닝 모델을 표준화된 환경에 배치.
- **결과:** 개발자와 데이터 과학자들이 동일한 환경에서 작업 가능해지며, 모델 학습 및 배포 과정이 효율적으로 운영됨.

지속적 통합 및 배포(CI/CD)를 위한 활용

사례

- MLOps 프로세스의 일환으로 CI/CD 파이프라인에 컨테이너 도입.

구체적 내용

- **문제 상황:** 머신러닝 모델의 빠른 개발, 테스트, 배포가 필요한 상황
- **해결 방법:** Jenkins, Travis CI와 같은 CI/CD 도구와 함께 Docker Container를 사용하여 코드의 빠른 테스트 및 배포 구현
- **결과:** 머신러닝 모델의 개발에서 배포까지의 시간 단축과 프로세스 자동화

5. MLOps에서 Container 인프라

MLOps에서 Container 인프라

중요성

- **MLOps에서의 중요성:** 컨테이너는 MLOps의 핵심 구성 요소로 자리 잡음. 데이터 과학자와 엔지니어가 동일한 환경에서 작업할 수 있게 해 주며, 모델의 개발, 테스트, 배포 과정을 효율적으로 만듦.
- **효율성과 확장성:** 컨테이너는 머신러닝 파이프라인의 효율성을 증가시키며, 클라우드 환경에서의 확장성을 제공. 이는 MLOps의 유연성과 속도를 증가시키는 데 기여
- **앞으로의 전망:** 컨테이너 기술은 계속해서 발전하고 있으며, MLOps의 미래에 큰 영향을 미침. 특히, 멀티 클라우드 및 하이브리드 클라우드 환경에서의 활용도가 증가할 것으로 예상.

6. Docker

Docker

Docker란?

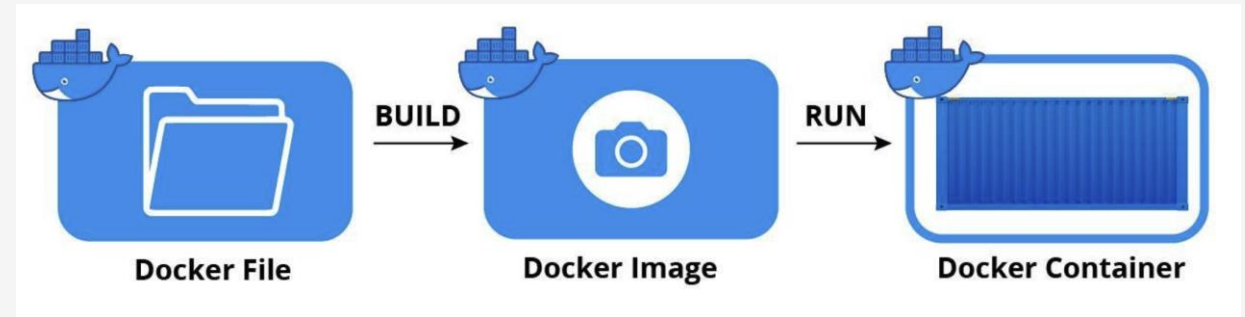
- **정의:** Docker는 컨테이너화 기술을 사용하여 애플리케이션을 패키징하고 배포하는 데 사용되는 오픈 소스 플랫폼. 이를 통해 애플리케이션을 쉽게 배포하고 실행할 수 있음
- **특징:** Docker는 경량화된 컨테이너를 제공하며, 이는 독립적이고, 이식성이 뛰어나며, 환경에 구애받지 않는 방식으로 애플리케이션을 실행



Docker

Docker 주요 구성 요소

- **Docker Image**
애플리케이션을 실행하는데 필요한 모든 파일과 설정을 포함하는 템플릿
- **Docker Container**
Image를 기반으로 실행되는 실행 Instance
- **Docker Daemon**
Docker Image와 Container를 관리하는 background service
- **Docker Registry**
외부 Image 저장소. 다른 사람들이 공유한 이미지를 사용할 수도 있고, private 하게도 저장 가능
- **Docker Client**
사용자가 Docker Daemon과 상호작용하는 Interface



Docker

Docker 네트워킹과 볼륨 관리

- Docker는 Container간 및 외부 네트워크와의 통신을 위한 다양한 네트워킹 옵션을 제공
- 예를 들어, bridge, host, overlay 네트워크 등이 있으며, 이는 다양한 상황에 맞게 컨테이너의 네트워크를 구성할 수 있음
- Docker는 데이터 영속성을 위해 볼륨을 사용하여 데이터를 저장할 수 있음. Container가 삭제되거나 재시작되어도 데이터를 유지할 수 있게 도와줌

Docker Compose

- 여러 Container를 정의하고 실행하기 위한 도구
- 복잡한 애플리케이션을 간단하게 정의하고 관리할 수 있게 도와줌